FactoryPilot, end to end

A spoken walkthrough of what the system does, who it is for, how a question actually travels through it, and what it deliberately refuses to do. Roughly 15 minutes read aloud.

Every screen in this script works on the deployed app. All fourteen Admin tiles were verified returning `200` after the eight that previously pointed at a development-only preview were rebuilt as real Fiori Elements applications. Nothing here is a mock-up and no tile is off-limits on camera.

Section 1 · ~90 seconds

01. The problem worth solving

A warehouse supervisor needs one number. How much of a material is on hand at a plant. Which goods movements were posted this morning. Whether an inventory count is still open.

That number exists. It is in S/4HANA, and it has been there all along. The difficulty is never that the data is missing — it is that getting to it costs a transaction code, a screen someone has to be trained on, a selection screen with fourteen fields, and often a colleague who knows which one of them matters.

So the number gets asked for instead of looked up. A message to a supervisor. A call to whoever is near a terminal. A spreadsheet that was accurate on Tuesday. The decision waits on a person, and the person is on the shop floor.

ON SCREEN Open on the empty chat with its four suggested questions. Do not type yet — let the viewer read them. **How much stock do we have**, **Show goods movements**, **What purchase orders are open**, **Which physical inventory counts are still open**.

FactoryPilot is a chat window over that same S/4HANA data. You ask in the words you already use. It goes and looks, and it shows you where it looked.

That last part is the one that matters, and most of this walkthrough is about it.

Section 2 · ~2 minutes

02. One question, all eight gates

Here is what happens between pressing enter and seeing an answer. Eight gates, in order. Two of them can answer without ever reaching SAP or a language model.

First, the *approuter*. The request carries an XSUAA token and a CSRF token. No token, no question — this is SAP's own front door, not something invented for this app.

Second, *user provisioning*. If this is the first time this person has ever asked anything, a user row is created for them. It grants nothing. It exists so that whoever hands out warehouse permissions can see that this person showed up and decide about them, rather than discovering them later in a log.

Third, *routing*. The question and the current load decide whether this deserves the heavy model or the light one. A stock lookup does not need a 120-billion-parameter model.

Fourth, *the quota gate*. Before any model or any SAP call, the user's allowance is checked and reserved. Someone over their limit costs nothing to refuse.

Fifth, *the cache*. Deliberately after the quota, never before. If the cache came first, anyone over their limit could pin one question and ask it free forever, and the quota would be unenforceable. So a cached answer is fast, but it still spends a request.

Sixth, *the agent loop*. The model is handed a catalogue of tools and decides what to do. More on that in a moment.

Seventh, *the cache write* — only on success, and only when no write is pending.

Eighth, *the audit row*. Every path writes exactly one, including both early exits. A refused question is as auditable as an answered one.

ON SCREEN Ask **How much stock do we have in plant 1710?** Let the thinking animation play — the mark breathes and the caption advances through **Reading your question**, **Querying SAP**, **Composing the answer**. Do not cut this; the visible stages are the point.

Section 3 · ~2 minutes

03. Nothing classifies your question but the model

There is no keyword list deciding what you meant. No regular expression looking for the word "stock". The model is handed a catalogue of tools, built fresh on every request from whichever business objects are registered and active, and it chooses one of three things.

It can answer directly. Say hello and it says hello — no SAP call, no data, and the answer is honestly marked as not grounded.

It can call a read tool. That becomes an OData query against S/4, and the rows come back into the loop for it to reason over. Up to eight rounds, and bounded by a clock as well as a count.

Or it can call a write tool — and that is where the loop stops, which is section six.

The consequence of building it this way is the one that sells it: adding a sixth business object is a configuration row naming an entity. Not a release. Not a code change. Not a conversation with a developer.

ON SCREEN Split screen. Left: the chat answering **hello** — instant, no sources, badge reads **Not grounded**. Right: the same window answering a stock question — sources present, badge reads **Grounded in SAP data**. Hold on the two badges.

The line to land here: "Not grounded" is not a failure. It is the system telling you it answered from the model and not from your data — which is correct for a greeting, and a red flag for a stock question. The badge is how you tell those apart at a glance.

Section 4 · ~2 minutes

04. Where the data actually comes from

Two backends, chosen per business object, and the model knows about neither.

Three objects read through **SAP Graph** on the customer's own Integration Suite tenant — material stock, material documents, and physical inventory. Graph puts one namespace in front of many S/4 services, so an object names an entity rather than a service path. Authentication is OAuth 2 client credentials against the tenant's own XSUAA.

Two objects — outbound deliveries and purchase orders — read through the SAP Business Accelerator Hub, because those entities are not in the Graph namespace. Pointing them at Graph would return nothing, and nothing reads exactly like "there are no purchase orders."

Material documents are the interesting case. Graph exposes document *headers*, but plant and material live on the *items*. So the query expands the association and filters inside the expansion, then flattens the result to one row per item — which means a document with three lines counts as three, not as one.

ON SCREEN Expand the **Sources** disclosure above an answer. Point at the system badge reading **SAP GRAPH**, the row count, the timing, and the full OData URL underneath. Then run `node scripts/graph-probe.js` in a terminal and show all three objects answering.

And that sources panel is the whole trust story in one control. It names the system, the number of rows, how long it took, and the exact filter that was sent. When an answer comes back empty, you can see whether the data is missing or whether the question went to the wrong plant. Those are very different problems, and without this panel they look identical.

Section 5 · ~90 seconds

05. The model layer, and why it is a chain

Model traffic walks an ordered list rather than depending on one provider.

Free OpenRouter models come first — each one its own rung. If a free model is rate-limited, the next thing tried is another *free* model, not the paid key. Only when the free tier is genuinely exhausted does it reach the customer's own OpenAI key. Below that sits an offline provider that computes answers from real tool output, so even with no model reachable at all the data stays true and only the phrasing gets plainer.

The models were not chosen from a marketing page. OpenRouter lists eighteen free models as supporting tool calling; probed with this application's own tool schema, fewer than half actually do. One of the largest returns nothing at all the moment tools are present. The ones in use were observed working, and the probe that established that ships with the project — because free availability shifts weekly and the next person deserves evidence rather than a guess.

Every hand-over is recorded. The audit row says which provider failed, and why.

ON SCREEN Terminal: `node scripts/openrouter-probe.js`. Let the USABLE and NOT USABLE verdicts scroll. This is a strong, cheap moment — it shows the engineering was empirical.

06. The write gate — the part that makes it safe

Reading data is reversible. Moving stock is not. So a write never executes inline, under any configuration.

When the model decides a goods movement is called for, the loop stops. It does not execute and then tell you. It produces a confirmation card describing exactly what would happen — material, quantity, from, to — and the run ends there. A separate, explicitly approved request is the only thing that can apply it.

There is a second rule that matters more than it sounds. If the model cannot fully describe the write, it does not propose one at all. An early version filled a missing quantity with one, and a missing material with a plausible-looking code. An operator could have approved a movement against a material nobody named. A confirmation card is only meaningful if every value on it came from the person about to approve it — so now it asks instead of guessing.

Approval policy layers on top: per user, per warehouse, per organisation, and the most restrictive layer wins. A permissive user policy cannot widen what the warehouse allows. Where policy demands it, the person who requested a movement cannot be the person who approves it.

ON SCREEN Type **Move 25 units of material FG500 from storage location 0001 to 0002 in plant 1010.** Show the confirmation card. Then — and this is the strongest single moment in the video — type an *incomplete* instruction like **move some stock to shipping** and show it asking for the missing values instead of inventing them.

07. Four roles, fifteen scopes

Authorisation is XSUAA — SAP's own, not a permissions table invented for this application. Fifteen scopes across eight services, gathered into four role collections that an administrator assigns in the BTP cockpit exactly as they would for any other SAP application.

Role collection	Who it is for	What it grants
FactoryPilot_BusinessUser	Supervisors, planners, operators — the people asking questions	Ask questions, and see their own token usage. Nothing else. No configuration, no other user's history.
FactoryPilot_ReadOnly	Auditors, analysts, anyone reviewing rather than operating	Read across configuration, tokens, users, audit, cache and integration — and change none of it.
FactoryPilot_ConfigAdmin	The person who connects systems and registers data	Register business objects, manage endpoints and cache policy. Deliberately

		cannot grant permissions or change quotas.
FactoryPilot_Administrator	The system owner	Ask questions, configure, and administer — the union of the operational and configuration roles.

The separation worth pointing out on camera is the third one. Whoever wires up a connection to S/4 is not automatically the person who decides who may move stock. Those are different jobs, and in this system they are different roles.

ON SCREEN BTP cockpit, Role Collections list, showing all four. Then the Users screen in the Admin app — reached from the working **Users** tile — showing per-warehouse read and write scopes.

Section 8 · ~2 minutes

08. Administration, and what the operator controls

Everything the system does is configuration, not code. Fourteen screens, each one a Fiori Elements application generated from the same CDS annotations that define the data — so the list, the filters and the detail page all came from one description rather than three.

Business Objects is the registry. Each active row becomes a tool the model can call — its entity, its filter template, the fields to select, and a prompt hint telling the model how to talk about it. This is the screen where a sixth data source gets added, and it is a form, not a deployment.

Integration Endpoints holds the connections. Kind, URL, authentication mode, timeout. Critically, it never holds a secret: it holds the *name* of the environment variable that holds the secret. The value lives in the platform, so an export of this table — or a backup, or an audit extract — carries no credentials.

Cache Policies decides how long an answer may be reused. Fifteen minutes by default; ten for deliveries, because delivery status moves through the day. And any question containing "today" or "now" has its lifetime clamped to midnight, so a "today" answer can never survive into tomorrow whatever the policy says.

Quota Policies caps spend per user, per role, or by default — daily, weekly and monthly. The shipped default is fifty questions a day, two hundred a week, five hundred a month.

Session Log is the record. One row per request: who asked, what they asked, which object answered, whether it was grounded, whether it came from cache, how many tokens it cost, and the URL that was actually called.

And eight more, for the people who have to run it

Agent Runs goes a level below the session log — the rounds inside a single turn, and every tool step underneath, with the URL and the duration of each.

Approvals is the queue of proposed writes: pending, approved, rejected, expired. Nothing moves

stock without passing through here.

Approval Policies is where the autonomy rules live — per user, per warehouse, per organisation, with a write ceiling and a second-approver flag.

Org Settings holds the organisation-wide defaults: default plant, the anomaly factor that decides when a quantity is unusual enough to need a human, and the digest hour.

Token Usage is one row per model call, and it flags an estimated count as estimated rather than letting it read as measured.

Model Routes is where the light, heavy and intent models are chosen, along with their fallbacks — this is the screen that shows the free models actually in use.

Cache Effectiveness reports hits, misses and tokens saved per object per day, so a cache lifetime can be judged rather than guessed at.

Connection Tests is the history of every connectivity check, with status and duration — the screen for "it worked on Monday".

ON SCREEN Open the Admin launchpad and let the KPI tiles load. Then open **Model Routes** — it shows the free models by name, which sets up section 5 — and **Agent Runs**, drilling into one run to show the tool steps underneath.

Section 9 · ~90 seconds

09. Security, stated plainly

Five things, and none of them are novel — which is the point. This is SAP's own security model, used as intended.

Identity is XSUAA. Single sign-on through the approuter. The application never sees a password and has no user store of its own.

Authorisation is scopes. Fifteen of them, enforced by the CAP framework at the service boundary rather than by application code remembering to check.

Secrets are never in the database and never in the repository. Configuration stores the name of an environment variable; the platform holds the value.

Writes require a human. Not a setting that can be turned off — the loop physically stops and returns a card.

Everything is audited. One row per request on every path, including refusals, plus the tool steps underneath it. A failed fetch is recorded as failed, and is deliberately kept out of the cache — so a wrong answer cannot outlive the outage that caused it.

ON SCREEN Session Log open, one row expanded. Then the Integration Endpoints form, pointing at the **Credential Reference** field — the name, not the secret.

Section 10 · ~90 seconds

10. Native to SAP, and what that buys

This is not an application that talks to SAP. It is an SAP application.

It is built on the SAP Cloud Application Programming model. It deploys to SAP Business Technology Platform as a standard multi-target application. Its front end is Fiori and follows the Horizon theme. Its identity is XSUAA. It reads S/4 through SAP Graph and the SAP Business Accelerator Hub. Its data lives in PostgreSQL on BTP.

What that buys is not architectural elegance — it is procurement and operations. There is no new vendor to assess, no data leaving the SAP estate to reach a backend that is not SAP's, no separate identity provider, and no second place to manage users. It is deployed, monitored, secured and audited by the same team and the same tools already running everything else on the platform.

Eight independently scoped services, so a customer can adopt or drop any one of them without touching the others.

Section 11 · ~90 seconds

11. What it costs

Be careful with this section. "Zero cost" is true of the current configuration and not of every configuration. The wording below is accurate; loosening it into "this is free" is the kind of claim a customer will test.

As it runs today, the running cost is nothing.

The models are free-tier OpenRouter models. The SAP Business Accelerator Hub sandbox is free. It is deployed on a BTP trial account. There is no per-seat licence and no per-question charge.

Three things keep it that way as usage grows. The **cache** means a repeated question costs no model call and no SAP call at all. The **quota** caps spend per person before anything is spent. And the **light and heavy routing** means a simple lookup does not pay for a large model.

The honest caveat: the paid OpenAI key is a fallback, and when the free tier is exhausted it does get used. That is a deliberate trade — an answer that costs a fraction of a cent is better than no answer during a demo. You control it, you can see it in the token usage, and the quota bounds it.

Section 12 · ~2 minutes

12. What it refuses to do

This is the section that will convince the sceptical people in the room, so give it time. Every one of these was a real failure at some point, and every one now has a test that fails if it comes back.

It will not execute a write inline. A goods movement needs a human between the request and SAP.

It will not propose a write it cannot fully describe. A missing quantity once defaulted to one, and a missing material to a plausible code.

It will not report a sample as a total. Twenty-five rows of a two-hundred-row result once answered with a confident figure that was about an eighth of the truth. A sampled answer now says it is a sample and quotes the real total.

It will not say "no records" when a fetch failed. That reads as "there is no stock" when the truth is "I could not check." A failed fetch is marked failed and kept out of the cache.

It will not guess your intent when the model is unavailable. Keyword matching once answered "what is your name" with an inventory count. The fallback now says the model is unavailable.

It will not exceed a quota under load. A cap of five once allowed all twelve simultaneous requests through. The limit is now part of the database update itself.

One hundred and sixty-nine tests across forty-two suites, and a good share of them exist to keep exactly these behaviours from coming back.

ON SCREEN Terminal: `npm test`. Let the summary land — **169 pass, 0 fail**. Hold for two seconds.

Section 13 · ~60 seconds

13. What is not built yet

Saying this out loud costs thirty seconds and buys more credibility than any feature in the video.

Photographs are not read. The camera button says so rather than collecting an image nothing looks at. The question pipeline is text-only.

Answers arrive complete rather than streaming word by word.

That is the whole list. Every other thing in this walkthrough — all fourteen administration screens included — is running against live SAP data today.

Section 14 · ~45 seconds

14. Closing

The demonstration is a chat window, and a chat window is easy to build. What is hard is everything behind it: knowing which system answered, being told when the figure is a sample, being stopped before a write, being unable to exceed a budget, and being able to see afterwards exactly what was asked and where the answer came from.

FactoryPilot is that second part. It happens to have a chat window on the front.

ON SCREEN End on an answered question with the **Sources** panel expanded — the URL visible, the row count visible, the grounded badge green. Hold.

Appendix · numbers you can quote

15. Verified figures

Every number below was checked against the running system rather than estimated. If a figure is not here, check it before you say it on camera.

Claim	Figure
CAP services, independently scoped	8
XSUAA scopes	15
Role collections	4
Registered business objects	5 — 3 via SAP Graph, 2 via Accelerator Hub
Entities available in the Graph namespace	7
Tests / suites	169 / 42, all passing
Default quota	50 per day · 200 per week · 500 per month
Cache lifetime	15 min default · 10 min deliveries · clamped to midnight for "today"
Question budget	75 s in the service, 120 s at the gateway
Agent rounds	up to 8, also bounded by the clock
Admin screens, all working	14

Written from the deployed system on 30 August 2026 · branch version2 Figures verified against the running app, the seed configuration and the test suite.